

WEEK-7

Task-1

Write a program that prints the addresses Of code, global, static, heap, and stack variables.
Analyze the Linux process address space layout and explain each memory segment.

Code:

```
GNU nano 8.7.1                                address.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function()
{
    printf("Code address    : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));

    if (heap_var == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    *heap_var = 40;

    printf("Memory Addresses:\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address    : %p\n", (void *)heap_var);
    printf("Stack address   : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/practical$ nano address.c
the_boss@DESKTOP-I8C8AJG:~/practical$ gcc address.c -o address
the_boss@DESKTOP-I8C8AJG:~/practical$ ./address
Memory Addresses:
Code address   : 0x6023184be1c9
Global address : 0x6023184c1010
Static address : 0x6023184c1014
Heap address   : 0x60231b8d9010
Stack address  : 0x7ffe05af31dc
the_boss@DESKTOP-I8C8AJG:~/practical$ |
```

Report: The program was successfully implemented to display addresses of different memory segments.

Addresses of code, global, static, heap, and stack variables were observed.

The results showed that different types of data are stored in different regions of virtual memory. Linux uses virtual address space and address randomization, so the exact addresses may change between executions.

Task-2: Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```
GNU nano 8.7.1 maps.c *
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process ID: %d\n", getpid());
    printf("Process is running...\n");
    printf("Check /proc/%d/maps from another terminal.\n", getpid());

    while (1)
    {
        sleep(5);
    }

    return 0;
}
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/practical$ cat /proc/988/maps
5d7e94159000-5d7e9415a000 r--p 00000000 08:30 26188
    /home/the_boss/practical/maps
5d7e9415a000-5d7e9415b000 r-xp 00001000 08:30 26188
    /home/the_boss/practical/maps
5d7e9415b000-5d7e9415c000 r--p 00002000 08:30 26188
    /home/the_boss/practical/maps
5d7e9415c000-5d7e9415d000 r--p 00002000 08:30 26188
    /home/the_boss/practical/maps
5d7e9415d000-5d7e9415e000 rw-p 00003000 08:30 26188
    /home/the_boss/practical/maps
5d7ec7527000-5d7ec7548000 rw-p 00000000 00:00 0
    [heap]
7ae712000000-7ae712028000 r--p 00000000 08:30 13647
    /usr/lib/x86_64-linux-gnu/libc.so.6
7ae712028000-7ae7121c0000 r-xp 00028000 08:30 13647
    /usr/lib/x86_64-linux-gnu/libc.so.6
7ae7121c0000-7ae71220e000 r--p 001c0000 08:30 13647
    /usr/lib/x86_64-linux-gnu/libc.so.6
7ae71220e000-7ae712212000 r--p 0020d000 08:30 13647
    /usr/lib/x86_64-linux-gnu/libc.so.6
7ae712212000-7ae712214000 rw-p 00211000 08:30 13647
    /usr/lib/x86_64-linux-gnu/libc.so.6
7ae712214000-7ae712221000 rw-p 00000000 00:00 0
7ae7123cf000-7ae7123d2000 rw-p 00000000 00:00 0
7ae7123d8000-7ae7123da000 rw-p 00000000 00:00 0
7ae7123da000-7ae7123de000 r--p 00000000 00:00 0
```

Report: The /proc/PID/maps file was used to study the virtual memory layout of a running process.

Different mappings such as code, data, heap, stack, and shared libraries were observed.

The permissions of each region were analyzed using read, write, and execute flags.

This experiment showed how Linux organizes and manages a process using virtual memory mappings.